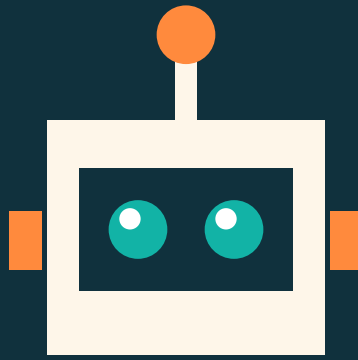




MateLab

Islas del Ingenio

Manual tecnico

Arquitectura, compilacion y mantenimiento

Aplicacion Android educativa de matematicas
para ninos de 8 a 12 anos

Version 1.0.0

24/08/2026

MateLab - Islas del Ingenio · v1.0.0

1. Ficha del proyecto

Concepto	Valor
Nombre	MateLab - Islas del Ingenio
applicationId	com.matelab.islas
versionName / versionCode	1.0.0 / 1
minSdk / targetSdk / compileSdk	24 / 34 / 34
Lenguaje	Kotlin 2.0.21
Interfaz	Jetpack Compose (BOM 2024.10.01), Material 3
Persistencia	Room 2.6.1 (SQLite)
Navegación	Navigation Compose 2.8.3
Asincronía	Coroutines 1.8.1, Flow / StateFlow
Serialización	kotlinx.serialization 1.7.3
Build	Gradle 8.9 (Kotlin DSL), AGP 8.5.2, KSP 2.0.21-1.0.28
JDK	17
Permisos	ninguno
Ficheros Kotlin	96 (86 de producción, 10 de prueba)
Líneas de Kotlin	~15.800

Sin Hilt, sin Firebase, sin backend, sin librerías de red ni de imágenes. Todas las versiones están fijadas en `gradle/libs.versions.toml`; no se usa ninguna versión dinámica.

2. Requisitos y compilación

2.1 Requisitos

- JDK 17 (Temurin recomendado).
- Android SDK con la plataforma 34 y build-tools 34.0.0.
- Gradle 8.9, o el wrapper una vez generado.

2.2 Generar el wrapper

El repositorio **no versiona** `gradle/wrapper/gradle-wrapper.jar` (es un binario). La primera vez:

```
gradle wrapper --gradle-version 8.9 --distribution-type bin
```

Ese comando descarga el `.jar` oficial y sustituye `gradlew` / `gradlew.bat` por los lanzadores estándar de Gradle.

2.3 Órdenes

```
./gradlew clean
./gradlew testDebugUnitTest
./gradlew lintDebug
./gradlew assembleDebug
./gradlew assembleRelease
```

Salidas:

- APK depuración: `app/build/outputs/apk/debug/app-debug.apk`
- APK producción: `app/build/outputs/apk/release/app-release.apk`
- Informe de pruebas: `app/build/reports/tests/testDebugUnitTest/index.html`

- Informe de lint: `app/build/reports/lint-results-debug.html`

La variante `*release*` activa R8 (`isMinifyEnabled`) y `shrinkResources`, y se firma con la clave de depuración para que el workflow pueda producir un APK instalable sin guardar secretos en el repositorio. **Para publicar en Google Play hay que sustituir esa firma por un keystore propio.**

2.4 Integración continua

`.github/workflows/android-build.yml` se dispara con cada push a main, con cada pull request y manualmente. Pasos:

1. `actions/checkout@v4`
2. JDK 17 (`actions/setup-java@v4`)
3. Android SDK con plataforma 34 (`android-actions/setup-android@v3`)
4. Gradle 8.9 (`gradle/actions/setup-gradle@v4`)
5. `gradle wrapper` para generar el binario que falta
6. `clean -> testDebugUnitTest -> lintDebug -> assembleDebug -> assembleRelease`
7. Renombrado de los APK, cálculo de **SHA-256** y generación de `deliverables/BUILD_RESULT.md`
8. Subida de artefactos: APK, informes de pruebas y de lint
9. Con una etiqueta `v*`, publicación de una `*Release*` con los APK adjuntos

El paso de lint usa `if: always()` para que se ejecute aunque fallen las pruebas y el informe quede disponible igualmente.

3. Arquitectura

3.1 Capas

```
com.matelab.islas
+-- MainActivity.kt           Única Activity; instala el splash
+-- core/
|   +-- MateLabApplication.kt Crea el AppContainer
|   +-- RootViewModel.kt     Perfil + ajustes; decide la ruta inicial
|   +-- di/AppContainer.kt    Inyección manual
+-- domain/                  Sin dependencias de Android
|   +-- model/               Content, Payloads, Progress, Runtime
|   +-- engine/              12 motores puros
|   +-- content/             Catálogo de contenido
|   +-- repository/          Interfaces
+-- data/
|   +-- local/               Room: entidades, DAO, semilla, mappers, JSON
|   +-- repository/          Implementaciones
+-- ui/
|   +-- theme/               Color, Type, Shape, Theme
|   +-- art/                 9 ficheros de ilustración con Canvas
|   +-- components/Componentes reutilizables y feedback (sonido + háptica)
|   +-- games/              12 mini-juegos + despachador
|   +-- screens/            12 pantallas con su ViewModel
|   +-- navigation/Grafo de navegación y helper de ViewModels
|   +-- audio/              SoundManager (SoundPool)
```

Regla de dependencias: `ui -> domain`, `data -> domain`. `domain` no conoce a nadie. `ui` nunca importa de `data` salvo dos utilidades sin estado (`MateJson` en pruebas y `PlayerRepositoryImpl.sanitizeAlias`, que es una función pura de la `companion`).

3.2 Inyección de dependencias

`AppContainer` crea la base de datos y los tres repositorios de forma perezosa. Se expone a `Compose` mediante `LocalAppContainer` y se consume con:

```
val viewModel: MapViewModel = mateViewModel { MapViewModel(it) }
```

`mateViewModel` es un helper inline sobre `viewModelFactory { initializer { } }` que evita escribir una fábrica por pantalla.

3.3 Flujo de estado

Cada `ViewModel` expone un `StateFlow<XUiState>` construido con `combine` sobre los `Flow` de Room, y la pantalla lo consume con `collectAsStateWithLifecycle()`. No hay `LiveData` ni estado mutable compartido.

4. Los motores de dominio

Los 12 objetos de `domain/engine` concentran toda la lógica evaluable. Son `object` de Kotlin sin estado ni dependencias de Android, por eso se prueban en la JVM sin emulador.

Motor	Responsabilidad	Algoritmo destacado
<code>GeoboardEngine</code>	Medir polígonos del geoplano	Fórmula del zapato para el área; test de intersección de segmentos por orientación para detectar polígonos cruzados
<code>FractionEngine</code>	Fracciones	<code>Fraction</code> comparable, con signo normalizado al numerador; equivalencia por producto cruzado en <code>Long</code> para no desbordar
<code>MeasureEngine</code>	Longitud, masa, capacidad, balanza	Conversión por factor a la unidad base; solución voraz de la balanza
<code>ClockEngine</code>	Reloj analógico	Hora deducida del ángulo (6°/min, 30°/h + 0,5°/min); aritmética modular de 12 h
<code>AngleEngine</code>	Ángulos	Normalización a [0,360), diferencia más corta con envolvente, clasificación
<code>SymmetryEngine</code>	Mosaicos	Índice reflejado, celdas que faltan y celdas que sobran
<code>PlaceValueEngine</code>	Base diez	Descomposición canónica y valor posicional
<code>PatternEngine</code>	Secuencias	Progresión aritmética/geométrica y unidad cíclica mínima para patrones visuales
<code>ShapeSortEngine</code>	Clasificador	Criterios sobre propiedades reales; validación de que el reto tiene solución
<code>ProgressEngine</code>	Progresión	Curva de niveles, estrellas, XP, desbloques y racha
<code>BadgeEngine</code>	Insignias	10 reglas evaluadas sobre un <code>BadgeContext</code>
<code>CollectibleEngine</code>	Objetos	Premio de misión y ocho hitos
<code>ReviewEngine</code>	Repaso	Construcción de la sesión por número de fallos y antigüedad

4.1 Curva de niveles

```
cumulativeXpFor(nivel) = 100·(nivel-1) + 25·(nivel-1)·(nivel-2)
```

Incrementos: 100, 150, 200, 250... hasta el nivel 30.

4.2 Estrellas

```
100 % de aciertos y sin pistas -> 3
100 % con pistas, o >= 75 %    -> 2
>= 50 %                        -> 1
resto                          -> 0
```

4.3 Experiencia

```
xp = aciertos · 10 · multiplicadorDificultad + estrellas · 5
```

con multiplicador 1 (Explorador), 2 (Aventurero) o 3 (Maestro).

5. Contenido: modelo y serialización

5.1 Modelo

World -> Mission -> Challenge. Cada Challenge lleva un ChallengePayload, una **clase sellada** con 12 subclases, una por mini-juego.

5.2 Serialización

El payload se guarda en la columna challenge.payload_json como JSON polimórfico de kotlinx.serialization, con discriminador "type":

```
{"type":"geoboard","grid":6,"objective":"AREA","target":4.0,
  "minVertices":3,"unitLabel":"cuadraditos"}
```

Ventaja: añadir un mini-juego nuevo no obliga a migrar el esquema de la base de datos, basta con una subclase más. MateJson.decodeOrNull devuelve null ante un JSON corrupto, y el mapper lo sustituye por un reto de continuación en vez de propagar la excepción.

5.3 Semilla

DatabaseSeeder vuelca el catálogo con SQL directo desde el callback de Room:

- onCreate: escribe catálogo y valores por defecto de perfil y ajustes.
- onOpen: si meta.catalog_version no coincide con Catalog.VERSION, o la tabla world está vacía, reescribe **solo el catálogo**. El progreso nunca se toca.

Se usa SQL directo, y no los DAO, para garantizar que el contenido está escrito antes de que la primera pantalla consulte la base.

Para publicar contenido nuevo: editar los ficheros Catalog*.kt y subir Catalog.VERSION.

6. Persistencia

13 tablas. Detalle completo en BASE_DE_DATOS.md y database/schema.sql.

- **Catálogo** (reescribible): world, mission, challenge, badge, collectible.
- **Jugador**: profile, settings, mission_progress, attempt, badge_unlock, collectible_unlock, review_item, meta.

Claves foráneas con ON DELETE CASCADE de world a mission y de mission a challenge. Índices en mission.world_id, challenge.mission_id, collectible.world_id y en attempt por mission_id, world_id y timestamp.

fallbackToDestructiveMigration() está activado a propósito: el contenido se regenera desde la semilla y una migración fallida nunca debe dejar la app inservible en el móvil de un niño. El coste es perder el progreso en un cambio de esquema, aceptable en una app sin cuenta.

6.1 Cierre de una misión

ProgressRepositoryImpl.finishMission es la operación central:

1. Inserta un attempt por reto y actualiza review_item.
2. Calcula estrellas y XP con ProgressEngine.
3. Fusiona el progreso conservando el mejor resultado histórico.
4. Suma XP al perfil y detecta subida de nivel e islas desbloqueadas.
5. Entrega el cristal de la misión si procede.
6. Evalúa insignias, después los hitos de colección, y vuelve a evaluar insignias (algunas dependen del tamaño de la colección).
7. Devuelve un MissionOutcome con todo lo ganado, que alimenta la pantalla de resultado.

7. Interfaz

7.1 Tema

MateLabTheme provee el esquema de Material 3 (claro y oscuro), la tipografía y tres CompositionLocal propios: LocalMateColors (paleta extendida), LocalReducedMotion y LocalBigText. paletteFor(WorldTheme) devuelve la gama de cada isla.

7.2 Ilustración

Todo el arte se dibuja en tiempo de ejecución con DrawScope. DrawUtils aporta las primitivas (polygonPath, starPath, wavePath, onCircle, diamondPath) y ArtRandom, un generador determinista tipo *xorshift*: la misma semilla produce siempre el mismo dibujo, así que los cristales no cambian de aspecto entre recomposiciones.

7.3 Mini-juegos

Contrato uniforme:

```
@Composable
fun XGame(payload: XPayload, enabled: Boolean, onSubmit: (Boolean) -> Unit,
    modifier: Modifier = Modifier)
```

Cada juego gestiona su propio estado y su botón *Comprobar*, y comunica el resultado con onSubmit. ChallengeGame despacha según el tipo del payload dentro de un key(challenge.id), de modo que el estado interno se reinicia limpio en cada reto.

Gestos: detectTapGestures y detectDragGestures sobre Modifier.pointerInput. El arrastre real del clasificador de figuras resuelve el destino comparando la posición de la figura con los rectángulos de las cestas mediante LayoutCoordinates.localBoundingBoxOf.

7.4 Navegación

Once rutas en Routes, dos con argumento (isla/{worldId}, mision/{missionId}). La ruta inicial la decide RootState.startRoute según el perfil: onboarding, creación de perfil o mapa. Mientras se lee el perfil se muestra un splash propio.

La misión no usa rutas separadas para informe y resultado: son tres fases (INFORME, JUGANDO, RESULTADO) de la misma pantalla, para no tener que propagar el MissionOutcome por argumentos de navegación.

7.5 Sonido y háptica

SoundManager carga seis WAV de res/raw en un SoundPool. UiFeedback combina sonido y HapticFeedback, respetando los ajustes mediante LocalSoundManager y LocalHapticsEnabled. No se usa el permiso VIBRATE porque View.performHapticFeedback no lo necesita.

Los efectos se generan con tools/gen_sounds.py, que usa solo la biblioteca estándar de Python (math, struct, wave) y aplica envolvente de ataque y caída para evitar chasquidos.

8. Pruebas

```
./gradlew testDebugUnitTest
```

171 pruebas en 10 clases, todas de JVM pura. CatalogIntegrityTest merece mención aparte: no prueba código, prueba **contenido**. Verifica que toda balanza tiene solución, que toda fracción pedida se puede pintar, que cada figura del clasificador encaja en exactamente una cesta, que las celdas de partida de los mosaicos están en la mitad correcta y que la respuesta marcada en cada patrón coincide con la que deduce PatternEngine.

8.1 Herramientas de verificación estática

Tres scripts de Python sin dependencias, útiles cuando no hay compilador a mano:

```
python tools/check_imports.py      # imports internos que no resuelven
python tools/check_structure.py    # llaves, recursos y despacho de mini-juegos
python tools/check_content.py      # límites de longitud y proporción de tests
```

9. Extender el proyecto

9.1 Añadir un reto

Editar el `Catalog*.kt` de la isla, añadir un `Challenge` con id único y subir `Catalog.VERSION`. Las pruebas de integridad validan el resto.

9.2 Añadir un mini-juego

1. Nueva subclase `@Serializable @SerializedName("...") data class XPayload` en `Payloads.kt`.
2. Nuevo valor en el enum `GameKind`.
3. Motor puro en `domain/engine` con sus pruebas.
4. Composable `XGame` en `ui/games` siguiendo el contrato.
5. Rama en `ChallengeGame` y etiqueta en `activityLabel`.
6. Retos que lo usen en el catálogo.

`tools/check_structure.py` avisa si falta la rama del despachador.

9.3 Añadir una isla

Nuevo `WorldTheme`, gama en `paletteFor`, dibujo en `IslandArt`, fichero `CatalogXxx.kt` y registro en `Catalog`. El mapa la coloca sola.

10. Decisiones y compromisos

Sin Hilt. Con tres repositorios, el contenedor manual es más legible y elimina el procesador de anotaciones más frágil de la cadena.

Arte por código, no por ficheros. Un APK sin bitmaps, escalable a cualquier densidad, y variantes generadas por semilla en vez de por duplicación.

Room sembrada con SQL directo. Garantiza que el contenido existe antes de la primera consulta, sin coordinar corrutinas con el ciclo de vida de la base.

Payload en JSON en vez de columnas por juego. Doce mini-juegos con parámetros dispares en columnas fijas darían una tabla llena de nulos.

Tres fases en una pantalla de misión. Evita serializar el `MissionOutcome` para pasarlo entre rutas.

Wrapper no versionado. Se evita meter un binario en el repositorio; el workflow lo regenera y el README documenta el paso manual.

11. Estado de compilación

El APK **no se compiló en la máquina de desarrollo**: el entorno tenía JDK 25, sin Gradle ni SDK de Android instalados. Lo que sí se ejecutó allí figura en `BUILD_REPORT.md`, junto con las instrucciones para obtener el informe real de GitHub Actions. No se ha simulado ningún resultado.